

GroupMemBench: Benchmarking LLM Agent Memory in Multi-Party Conversations

Jingbo Yang*
UC Santa Barbara

Kwei-Herng Lai*
Microsoft

Xiaowen Wang
Microsoft

Shiyu Chang
UC Santa Barbara

Yaar Harari
Microsoft

Evgeniy Gabrilovich
Microsoft

Abstract

Large Language Model (LLM) agents increasingly serve as personal assistants and workplace collaborators, where their utility depends on memory systems that extract, retrieve, and apply information across long-running conversations. However, both existing memory systems and benchmarks are built around the dyadic, single-user setup, even though real deployments routinely span groups and channels with multiple users interacting with the agent and with each other. This mismatch leaves three properties of group memory unmeasured: (i) group dynamics that go beyond concatenated one-on-one chats, (ii) speaker-grounded belief tracking, where the per-user memory modeling is needed, and (iii) audience-adapted language, where Theory-of-Mind shifts produce role-specific vocabulary. We introduce **Group-MemBench**, a benchmark that exposes all three. A graph-grounded synthesis pipeline produces multi-party conversations with controllable reply structure and conditions each message on per-user personas and target audiences. An adversarial query pipeline then binds every question to a specific asker across six categories, spanning multi-hop reasoning, knowledge update, term ambiguity, user-implicit reasoning, temporal reasoning, and abstention, and iteratively searches challenging, realistic queries that reflect comprehensive memory capability. Benchmarking leading memory systems exposes a sharp collapse: the strongest one reaches only 46.0% average accuracy, with knowledge update at 27.1% and term ambiguity at 37.7%, while a simple BM25 baseline matches or exceeds most agent memory systems. This indicates current memory ingestion erases the structural and lexical features group memory depends on, leaving multi-user memory far from solved.

 [Dataset](#)  [Code](#)

1 Introduction

Large Language Model (LLM)-powered agents have demonstrated strong capabilities across diverse tasks, both as personal assistants [1–5] and as workplace collaborators [6]. To handle the conversational history that accumulates over long-horizon use, modern agents rely on memory mechanisms for extracting, consolidating, and retrieving information from past interactions. In many realistic scenarios, however, agents are deployed in group communication contexts (*e.g.*, channels, threads, and project spaces), where multiple users interact with the same agent and with each other over time. Despite this, current agent memory systems [7–9] and their evaluations [10–13] almost exclusively target the dyadic, single-user setting, leaving the multi-party memory dynamics largely unmeasured.

*Equal Contribution. Correspondence to: jingbo@ucsb.edu, henrylai@microsoft.com.

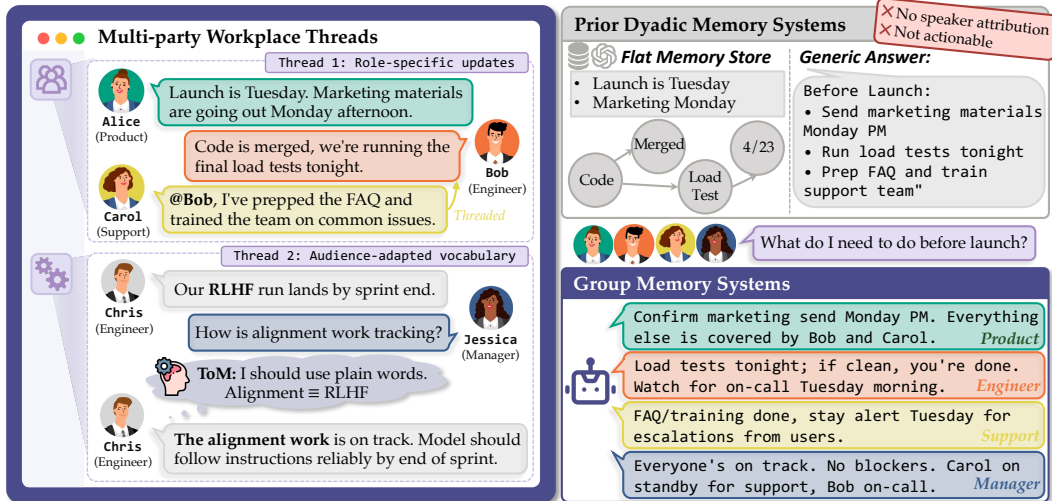


Figure 1: Dyadic memory systems are inadequate for group memory, which demands joint modeling of the collective task flow and per-user memory states. The multi-party attributes, such as Theory-of-Mind, also make this fundamentally harder than a multi-user extension of dyadic memory.

The transition from single-user memory to multi-party group memory introduces qualitatively new complexities along three dimensions. ① **Group dynamics.** Single-user interactions typically follow a straightforward instruction, i.e., response paradigm, whereas multi-party dialogues encompass richer social patterns such as open discussions, threaded debates, and collaborative consensus-building. The collective task flow therefore cannot be reduced to a flat sequence of turns or a set of parallel one-on-one streams. ② **Speaker-grounded belief tracking.** The identity of the message author becomes part of its content: identical statements from different speakers may warrant entirely different memory updates, and identical queries from different askers may warrant different answers. An agent must therefore contextualize its reasoning around the specific user’s preferences, role, and historical knowledge, rather than treating the conversation as an unattributed text stream. ③ **Audience-adapted language.** Human speakers naturally exhibit Theory of Mind (ToM) [14–17], adapting their wording to assumed shared knowledge with their audience. In workplace settings this surfaces as peer-specific jargon. For example, "token" may denote a unit of text in NLP discussions and an authentication credential in system-design discussions. This produces systematic lexical shifts across roles that pose a direct challenge to retrieval-augmented generation [18, 19].

To address the complexities, we benchmark existing memory mechanisms in multi-party group settings for a central question: *can current memory systems serve the right user, with the right belief, under the right interpretation, when memory is built from group conversations?* We decompose this into three sub-questions, each paired with a benchmark mechanism designed to isolate the corresponding source of difficulty. (Q1) **Can memory systems capture group dynamics that go beyond concatenated one-on-one chats?** We generate conversations with a controllable reply structure that induces threaded debates, multi-user replies, and cross-topic shifts, rather than a flat turn sequence. (Q2) **Can memory systems condition extraction, consolidation, and retrieval on speaker identity?** We bind each query to a specific asking user and introduce three speaker-aware adversarial categories: *user-implicit reasoning* hides the asker behind first-person references, *knowledge update* maintains incompatible preferences across users, and *term ambiguity* makes a term’s referent depend on who is asking. (Q3) **Can memory systems remain robust to ToM-driven lexical shifts across roles?** We condition messages on per-user personas and explicit target audiences so that the same underlying fact appears in role-specific wording; *term ambiguity* then asks questions in the asker’s vocabulary while the supporting evidence is recorded in another role’s phrasing, creating a controlled retrieval stress test. It is worth mentioning that benchmark difficulty is enforced rather than asserted: each accepted query must defeat a competent retrieval baseline through a Solve-Judge-Refine loop (Section 3.3.1).

Together these design choices yield **GroupMemBench** (Figure 1). Benchmarking leading memory systems on it reveals systematic gaps in group memory: even the best-performing system achieves

only 46.0% average accuracy. Abstention is largely preserved (above 80%), but content-bearing categories such as *knowledge update* and *term ambiguity* drop to 27.1% and 37.7%. We expect progress on GroupMemBench to reflect improved speaker-conditioned retrieval, belief tracking across users, and structure-aware retrieval in multi-party conversations.

To summarize, we make the following contributions in this paper:

- We define group-memory evaluation along three axes: dynamics, speaker grounding, audience adaptation.
- We introduce **GroupMemBench**: multi-party conversations with user-bound queries.
- We construct the benchmark via reply-structure control, speaker-aware categories, and persona/audience conditioning.
- We enforce difficulty with Solve-Judge-Refine and benchmark leading memory systems.

Table 1: Comparison of GroupMemBench with representative long-term memory benchmarks. **Multi-User**: dialogues feature multiple human-modeled speakers. **ToM Adaptation**: speakers adapt vocabulary and explanations based on the roles and prior knowledge. **Threaded**: messages form a post-and-reply hierarchy rather than a flat turn-by-turn timeline. **User-Cond. Q.**: the gold answer can change with the querying user’s identity. **# Topics**: number of distinct conversation topics organized in the corpus or sub-task types. **# Turns**: total messages/utterances in the corpus; “—” indicates the benchmark does not report a turn-level statistic. **Avg. Length**: average tokens per dialogue (values taken from [11]). **Eval. Dimensions**: **Acc** = answer accuracy; **Cost** = LLM and embedding inference cost incurred while constructing memory; **Storage** = processed memory overhead.

Benchmark	Multi-User	ToM Adaptation	Threaded	User-Cond. Q.	# Topics	# Turns	Avg. Length	Eval. Dimensions
LoCoMo [20]	✗	✗	✗	✗	1	15K	9K	Acc
LongMemEval [13]	✗	✗	✗	✗	6	11K	115K	Acc
MemoryAgentBench [21]	✗	✗	✗	✗	4	—	100K–500K	Acc Cost
MemoryBench [12]	✗	✗	✗	✗	3	—	30K–380K	Acc Cost
EverMemBench [22]	✓	✗	✗	✗	15	51K	845K	Acc
GroupMemBench (Ours)	✓	✓	✓	✓	123	120K	2.7M	Acc Cost Storage

2 Related Work

Agent Memory Systems. Agent memory systems persist useful facts across long conversations so that an LLM agent can answer later queries without re-processing the full history [23–27]. Mainstream designs cast ingestion as online fact extraction over a structured store: MemGPT [28] manages a tiered working/archival memory with explicit paging, Mem0 [9] extracts atomic facts and applies add/update/delete operations during ingestion, A-Mem [29] links extracted notes into a self-organizing graph, HippoRAG [19] grounds dense retrieval in an entity-centric knowledge graph for multi-hop recall, and Hindsight [8] adds a reflection stage that rewrites consolidated memory over time. A recent position paper [7] surveys this landscape and argues for richer procedural memory. All of these systems were designed around dyadic interactions and inherit two structural assumptions that break in group settings: `user_id` is used as an isolated namespace that separates data stores rather than as a semantic attribute that conditions extraction and retrieval, and conflicting statements are resolved by overwriting older entries rather than preserving coexisting beliefs from different speakers. GroupMemBench specifically probes these two assumptions by requiring systems to attribute each fact to its speaker and to keep **concurrent** preferences addressable by the querying user.

Long-term Memory Benchmarks. Long-term memory benchmarks have scaled rapidly along context length, session count, and reasoning depth while keeping the interaction format strictly dyadic. LoCoMo [20] and LongMemEval [13] expand conversations to hundreds of sessions and test multi-session recall; MemBench [30], MemoryAgentBench [21], MemoryArena [10], AMA-Bench [11], and MemoryBench [12] broaden the question taxonomy to include multi-hop reasoning, temporal alignment, knowledge updates, and continual learning. Across this line of work, however, each utterance has a single author and every query is issued by “the user”: there is no notion of speaker-conditioned retrieval, coexisting preferences, or role-dependent vocabulary, so memory systems tuned

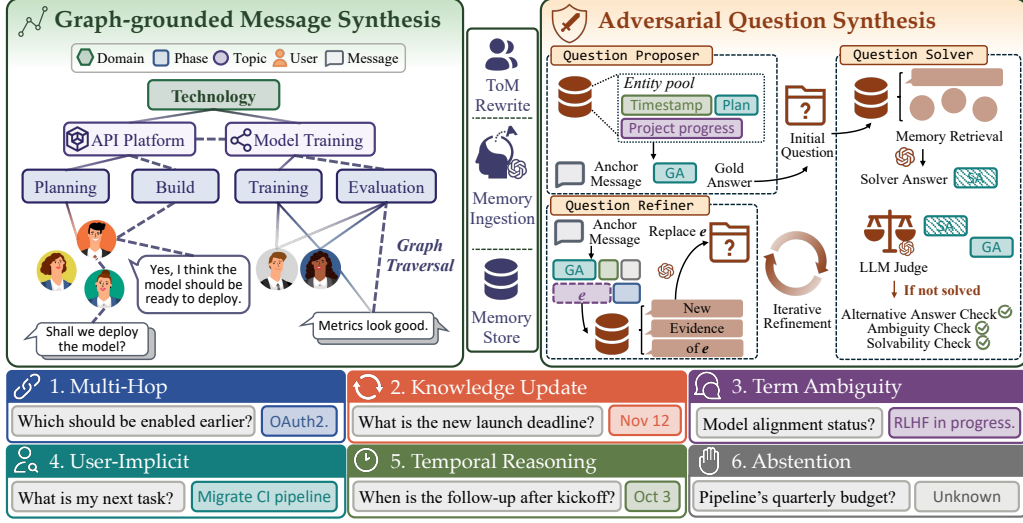


Figure 2: Overview of the GroupMemBench data synthesis pipeline .

on these benchmarks are never forced to model *who* said what to *whom*. EverMemBench [22] is the closest prior effort to evaluate memory under multi-party dialogue, yet its speakers are **homogeneous** chat participants without persona or Theory-of-Mind modeling, leaving agents untested on user-implicit reasoning and on vocabulary that shifts with the **addressee**. GroupMemBench fills this gap by pairing graph-grounded multi-user synthesis with adversarial queries that explicitly target speaker-dependent answers.

3 GroupMemBench

3.1 Problem Formulation

Group Memory Environment Let $\mathcal{U} = \{u_1, \dots, u_K\}$ denote the set of K users in a group, and let u_a denote the memory-augmented agent. The conversation is a **chronologically** ordered sequence $\mathcal{C} = (m_1, \dots, m_N)$, where each message is a tuple $m_i = (a_i, t_i, c_i)$ with author $a_i \in \mathcal{U} \cup \{u_a\}$, timestamp t_i , and textual content c_i . Each m_i may correspond to either a user–user interaction or a user–agent interaction, and every message, including those authored by u_a , enters the shared history and conditions all subsequent turns, so the agent’s replies evolve as a Markov decision process over $\mathcal{C}$. This contrasts with the dyadic setting ($K = 1$, i.e., one user plus the agent), where a_i trivially alternates and the author tag carries no information.

Starting from an empty state $\mathcal{M}_0$, the agent maintains a memory $\mathcal{M}_i$ that is incrementally updated after each message: $\mathcal{M}_i = f_{\text{update}}(\mathcal{M}_{i-1}, m_i)$. The update function f_{update} is left abstract, as concrete implementations vary across memory systems (e.g., fact extraction, graph updates, or raw append).

Task Definition We evaluate the memory state through memory-grounded question answering. After observing a prefix $\mathcal{C}_{1:T}$ and forming memory $\mathcal{M}_T$, the agent receives a query q issued by a specific user $u_q \in \mathcal{U}$ (with $q \in \{1, \dots, K\}$), and produces an answer $a = f_{\text{ans}}(q, u_q, \mathcal{M}_T)$, which is itself appended to $\mathcal{C}$ as the next message and thereby updates $\mathcal{M}_{T+1}$ and shapes the future trajectory of the conversation. The explicit dependence on u_q is the key departure from dyadic formulations, where u_q is a constant and is typically dropped. In the group setting, the ground-truth answer may differ across querying users even for the same q : resolving references in q , interpreting vocabulary, and ranking retrieved facts all depend on u_q ’s identity, role, and prior history. A correct system must therefore treat u_q as a first-class input, not an implicit context.

3.2 Synthesizing Multi-Party Conversations

We generate multi-party conversations by separating structure from text. A typed graph encodes domains, topics, phases, and users, and is augmented during generation with message nodes connected

by reply-to and authored-by links. Each new message begins with a graph traversal that selects the author, recipients, and thread/phase context; a language model then realizes the message conditioned on the retrieved situational history. This factorization makes group-level phenomena controllable while maintaining long-horizon coherence.

Knowledge Graph Representation. We represent the conversational environment as a directed graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ that grows over time. The structural backbone $\mathcal{V}_0$ contains four node types: domains (d), topics (τ), phases (ϕ), and users (u). The graph $\mathcal{G}$ is augmented during generation with a fifth type, messages (m), as they are produced. Each phase node carries an owner, a status drawn from a category-specific schema (RISKS, MILESTONE, or WORK ITEMS, each with its own status progression), a target date, and a temporal window over which messages can be timestamped. Each user node carries a contextual profile $\mathcal{C}_u$ that records persona attributes (role, tone, style, expertise) and prior message history; tone and style combinations are drawn without replacement so every user has a distinct voice. The edge set $\mathcal{E}$ encodes hierarchical relations ($d \xrightarrow{\text{has_topic}} \tau, \tau \xrightarrow{\text{has_phase}} \phi$), membership relations ($u \leftrightarrow d, u \leftrightarrow \phi$), cross-domain bridges via shared categories ($d \xrightarrow{\text{related_to}} d'$), and, once messages exist, threading and authorship relations ($m \xrightarrow{\text{reply_to}} m', m \xrightarrow{\text{authored_by}} u$). Grounding generation in $\mathcal{G}$ allows messages to reason over user-specific trajectories, phase progress, and inter-project dependencies rather than being conditioned on isolated prompts. The full schema is given in Table 4, Appendix A.

Path-Based Context Sampling. Each new message is generated by sampling a traversal path over $\mathcal{G}$ that determines the message’s author, recipients, and conversational situation. We define three path templates corresponding to the primary dynamics of group dialogue, and sample among them with mixture weights (0.2, 0.6, 0.2):

- **Channel post** ($d \rightarrow \tau \rightarrow \phi \rightsquigarrow u$): opens a new sub-thread by walking from a domain through one of its topics to an active phase, biased by a *post-need* weight that favours phases with few or no posts, and then sampling an author from the phase’s active members.
- **Threaded reply** ($d \rightarrow \tau \rightarrow \phi \rightsquigarrow m \rightarrow u$): walks to a phase under a *phase-heat* weight (favouring phases with recent activity and unanswered posts), then selects a parent message m from the phase’s history via **preferential** attachment combined with linear recency decay and a post-over-reply bias, and finally chooses a responder distinct from the parent author and the last few speakers.
- **Cross-project bridge** ($u \rightarrow \phi \rightarrow \tau \rightarrow d \xrightarrow{\text{rel.}} d' \rightarrow \tau' \rightarrow \phi' \rightarrow u'$): anchors at a user u , follows the *related_to* edge induced by shared domain categories to a sibling project d' , and lands on a target user u' with the same role as u in a temporally consistent phase ϕ' , capturing cross-team continuity such as role-to-role consultations.

A sampled path yields a set of contextual nodes that jointly define the generation condition. From these nodes we retrieve the corresponding metadata and the most recent messages within a fixed-size context window, providing the local turn-by-turn history. To prevent the long, narrow phases from devolving into paraphrastic loops, we additionally maintain a per-phase memory: at first use of a phase, we prompt the LLM once to **enumerate** a small budget of distinct sub-issues that a real team would discuss, and every few messages we refresh a short rolling summary of which points are already **DECIDED**, which questions remain **OPEN**, and which commitments are still **PENDING**. This summary, together with the active sub-issue, is appended to the retrieved history to form the situational context $\mathcal{S}_u$ used at the current generation step. Sampling probabilities, weight functions, and graph-walk details are provided in Appendix B.

Message Generation. Given the situational context $\mathcal{S}_u$, the user profile $\mathcal{C}_u$, and the path type, we construct a prompt that encodes the author’s persona, the current topic and phase, the phase’s stage (*early*, *middle*, or *late*, derived from a Poisson-bursty timestamp schedule) and status goal, the retrieved context, and the intended conversational scenario (full templates in Appendix C). The prompt is passed to a language model (GPT-5 in our implementation) to sample a candidate message

$$\hat{m} \sim p(\cdot \mid \text{prompt}(\mathcal{S}_u, \mathcal{C}_u, \text{stage}, \text{path})). \quad (1)$$

For replies, the prompt explicitly references the parent message to enforce threading coherence. The accepted message is instantiated as a new node in $\mathcal{G}$, annotated with author, timestamp, and path

type, and linked to the relevant users, phase, and parent message. To keep the corpus from being artificially clean, the policy layer stochastically injects four classes of realism perturbations: (i) *noise messages* ($\sim 5\%$) that contain a subtle factual mistake, stale reference, or off-topic drift; (ii) *disagreement turns* ($\sim 25\%$ of replies) that push back on or conditionally accept a prior point; (iii) *stylistic imperfections* ($\sim 20\%$) such as typos, fragments, or self-corrections, alongside ultra-short replies and tone jitter; and (iv) *decision changes* ($\sim 5\%$ of eligible phases) in which a previously committed decision is deliberately reversed mid-phase, with the reversal recorded as ground-truth metadata for downstream evaluation. Using this pipeline, we synthesize group conversations spanning four domains (Technology, Healthcare, Manufacturing, Finance), with each message labeled as either an initiating *post* or a threaded *reply*. Per-domain corpus statistics are reported in Appendix F.

Quality Assessment. We adapt G-Eval [31] to the group-chat setting and assess synthesized dialogues along six dimensions chosen to reflect properties specific to multi-party interaction: *Naturalness*, *Coherence*, *Diversity*, *Contextual Relevance*, *Momentum*, and *Engagingness*, each scored from 0 to 5 by GPT-5 as judge (rubric in Appendix E). We compare our four synthesized domains against two references: a *Single-Prompt Baseline*, which generates group messages from a single monolithic prompt without graph grounding, and a *Real-World Upper Bound* drawn from authentic group chat logs of our own in 1 month horizon. Each score is averaged over 10 seeds at fixed temperature. As shown in Figure 3, our synthesis matches the real-world conversations on most dimensions and substantially outperforms the single-prompt baseline, confirming that graph-guided generation yields conversations that are structurally and stylistically realistic.

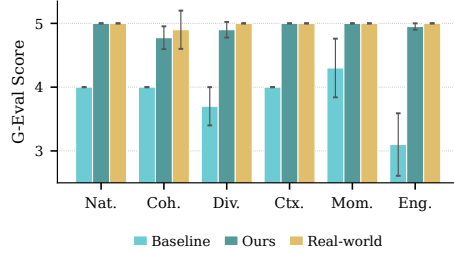


Figure 3: G-Eval scores across six dimensions. Our graph-guided synthesis (four domains) closely tracks the real-world upper bound and substantially outperforms the single-prompt baseline. Scores averaged over 10 seeds; shaded bands indicate ± 1 std.

3.3 Adversarial Query Construction

To systematically evaluate distinct failure modes in memory-augmented agents, we synthesize queries across six orthogonal dimensions. We employ a shared adversarial pipeline that decouples benchmark difficulty from human labeling effort: a query is only accepted into the benchmark if a competent retrieval-based baseline empirically fails to answer it.

3.3.1 Shared Generation Pipeline

Given a multi-party conversation, each query is generated through a rigorous four-stage process: **① Anchor Selection:** A lightweight filter identifies candidate messages containing the target entities, which an LLM classifier then verifies for concrete answerability. **② Initial Composition:** A type-specific question proposer LLM drafts a natural language question, based on the anchor message as the initial evidence and target entity as the gold answer. **③ Solve–Judge–Refine Loop:** A baseline agent attempts the question using retrieved context. An LLM judge evaluates the response. If the solver succeeds, the query is iteratively refined to increase difficulty. If it fails, the query is accepted. **④ Output:** For each accepted query, we retain the final question, gold answer, querying user ID, and a per-round evidence trace.

3.3.2 Dimensions and Type-Specific Strategies

Within this *harness*, we instantiate six query types, each targeting a distinct capability of group memory; type-specific generation and validation details are deferred to Appendix C.

- **Multi-Hop Reasoning:** Tests the agent’s ability to chain evidence scattered across distant messages and threads, where the answer is reachable only by linking a target entity to a separate “*pivot*” message that refers to it indirectly.
- **Knowledge Update:** Tests whether the agent tracks dynamic state changes and prefers the latest consensus over outdated facts. Each query is anchored to a documented decision change (*e.g.*, a rescheduled deadline) and may probe the current state, the trigger, or the prior value.

Table 2: Accuracy (%) of memory systems on GroupMemBench across six query categories. Best per column in **bold**, second-best underlined.

Method	Multi-Hop	Update	Ambiguity	Implicit	Temporal	Abstention	Average
<i>RAG-based Methods</i>							
BM25	<u>40.11</u>	<u>25.23</u>	14.15	40.82	54.94	<u>77.98</u>	<u>43.22</u>
text-embedding-3-large	36.26	23.36	21.70	46.94	<u>32.72</u>	75.23	38.04
GraphRAG [32]	12.09	14.02	19.81	14.29	5.56	66.97	20.56
<i>Agent Memory Systems</i>							
Mem0 [9]	21.98	4.67	11.32	20.41	16.67	82.57	25.73
MemGPT [28]	22.53	17.76	20.75	28.57	12.42	<u>77.98</u>	28.15
A-Mem [29]	35.16	22.43	26.42	46.94	23.46	67.89	35.10
HippoRAG [19]	39.56	27.10	<u>30.19</u>	<u>42.86</u>	29.63	75.23	39.72
Hindsight [8]	42.31	17.76	37.74	40.82	54.94	77.06	46.01

- **Term Ambiguity:** Tests robustness to speaker-conditioned vocabulary. The question is phrased in the asker’s role-specific terminology while the supporting evidence uses a different role’s variant (*e.g.*, engineer’s “PPO” vs. manager’s “alignment”), so the agent must resolve referents through understanding and linking the memories from different speakers, not the surface form.
- **User-Implicit Reasoning:** Tests whether the agent can resolve first-person references to a specific user. Queries are cast in the first person, emphasizing memory modeling of long-term user profile.
- **Temporal Reasoning:** Tests chronological reasoning over the dialogue. Questions are grounded on both relative time expressions (*e.g.*, “by EOD this Friday”) and timestamps attached to messages.
- **Abstention:** Tests resistance to hallucination at the edge of the agent’s knowledge. The query targets a fact absent from the conversation, requiring the agent to refuse rather than **fabricate**.

4 Experiments

4.1 Experimental Setup

Baselines. We compare against two families of memory approaches that represent the dominant paradigms in the current literature. ❶ **retrieval-augmented generation (RAG)** baselines, which treat the raw conversation as a passive document store and retrieve relevant chunks at query time. We include regular RAG with different retrievers, and graph RAG, which augments dense retrieval with an entity-centric knowledge graph. ❷ **agent memory systems** that actively extract, consolidate, and index facts during ingestion: *MemGPT* [28], *Mem0* [9], *A-Mem* [29], etc.

Evaluation Metrics. (1) **Accuracy:** question-answering accuracy judged by GPT-5 against reference answers [31], reported per category and averaged. We validate the LLM judge reliability on a human-annotated subset (Appendix I). (2) **Efficiency:** *ingestion cost* (USD) sums all LLM and embedding calls during memory construction, with query-time calls excluded; *storage footprint* (GB) is the on-disk size of each baseline’s per-domain store. Token-to-USD pricing, the proxy used for token accounting, and the procedure for apportioning shared databases are detailed in Appendix H.

Implementation Details. To isolate methodological differences from backbone differences, we standardize the underlying models across all baselines that do not ship with pretrained checkpoints. During *memory ingestion*—including fact extraction, summarization, and graph construction—all systems use gpt-4o-mini as the backbone language model. At *query time*, the answering agent uses GPT-5, and all dense retrieval components use OpenAI’s text-embedding-3-large. All baselines are evaluated following the official implementations.

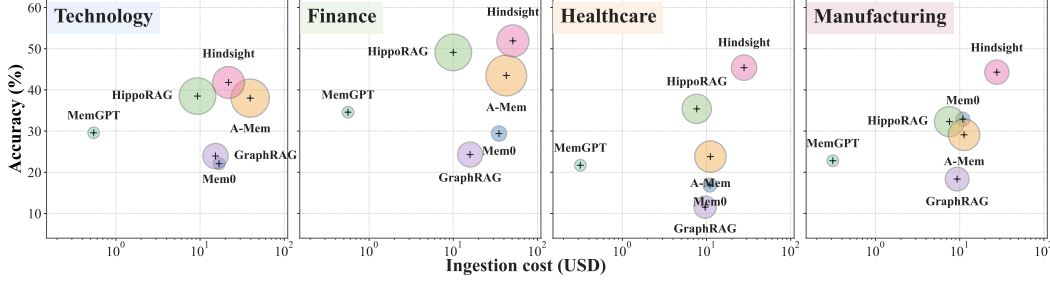


Figure 4: Performance–efficiency trade-off across the four domains. Each marker is one of six baselines (five agent memory systems plus GRAPHRAG): x -axis is ingestion cost (USD, log scale), y -axis is per-domain QA accuracy (%), and bubble area encodes on-disk storage in GB. Per-(baseline, domain) numerical values are listed in Table 11.

4.2 Main Results

Table 2 reports cross-domain accuracy across the six query categories (per-domain numbers in Appendix G). We organize the discussion around the three questions raised in the Introduction: across baselines, current memory systems (**Q1**) flatten group structure during ingestion, (**Q2**) fail to condition extraction and retrieval on speaker identity, and (**Q3**) cannot resolve lexical shifts even after the gold message is retrieved. The two categories where systems perform best, Temporal and Abstention, are also the ones least dependent on multi-party structure: Temporal answers are pinned to a date, and Abstention only asks whether the answer is in memory at all.

(Q1) Group structure is flattened during ingestion. No system exceeds 50% on Multi-Hop, the category that most directly probes evidence chained across thread structure. The deeper signal is that BM25, with no semantic processing during ingestion, beats four of the five agent memory systems on average J. Agent memory was designed to compress one user’s history into self-contained facts, and that compression discards exactly the structure the multi-party setting needs.

(Q2) Memory systems do not condition on the speaker. Three categories isolate this gap. On *User-Implicit*, the best agent memory system matches a bare text-embedding-3-large retriever, indicating no real per-user modeling and the extra ingestion machinery contributes nothing here. On *Knowledge Update*, where decisions usually shift through back-and-forth among multiple users, no system passes 28%; Mem0 collapses to 4.67 because its extractor appends new statements alongside obsolete ones without speaker-conditioned consolidation. *Term Ambiguity* reinforces the picture: dyadic memory has no notion of per-speaker referents, so the ambiguity caused by a word like “token” is irrecoverable. The two cross-category outliers tell the same story: Mem0’s high Abstention is conservative storage rather than calibrated refusal, and Hindsight’s gains on Multi-Hop and Ambiguity come from aggregating paraphrased notes across messages, the same aggregation that locks old decisions and drives its Update score below 18%.

(Q3) Lexical shifts are the residual failure even when retrieval succeeds. Term Ambiguity stays below 38% across all baselines, and the gap persists even after factoring out retriever quality: as we show in Section 4.4, $P(\text{correct} \mid \text{gold retrieved})$ remains uniformly low for this category, making it the only type where representation, not retrieval, is the wall. As these lexical shifts naturally attack the retrieval process, this further raises a future research direction on how to handle multi-party cognitive differences. We believe the solution to this should be a collaborative evolution of both memory modeling (*e.g.*, agent memory ingestion) and agent reasoning improvement.

4.3 Performance-Efficiency Trade-off

Per-domain ingestion cost varies by nearly two orders of magnitude across baselines (\$0.31–\$51.04), and on-disk storage spans 0.69–8.63 GB. Figure 4 answers whether this expense translates into accuracy; per-(baseline, domain) values are tabulated in Appendix H.

Most baselines are Pareto-dominated by HippoRAG. HIPPO-RAG sits on the cost–accuracy Pareto front in all four domains; A-MEM, MEM0, and GRAPH-RAG are dominated in three or four of them. The GRAPH-RAG case is informative: despite sharing similar “graph + RAG” structure as HippoRAG, it costs more and trails by 14–25 absolute accuracy points, indicating that the entity-centric graph

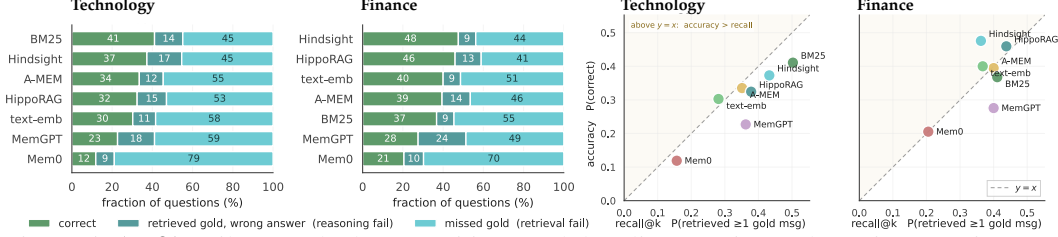


Figure 5: **(Left)** Failure-mode decomposition: each baseline’s 185 non-abstention questions per domain split into *correct*, *reasoning* failure, and *retrieval* failure. **(Right)** Retrieval recall vs. answer accuracy. Markers *on* the diagonal are retrieval-bottlenecked; *below* the diagonal indicates reasoning loss (gold surfaced but answered wrong); *above* indicates the system answered correctly without the gold message, typically by aggregating other ingested context.

construction in HippoRAG is the load-bearing design choice rather than graph augmentation in the abstract. The two ends of the front are MEMGPT (cheap embedding-only ingest, accuracy capped at 22–35%) and HINDSIGHT (highest accuracy at 2–5× HippoRAG’s cost), where the cost premium buys only 2–12 absolute accuracy points, concentrated on the smaller corpora.

Storage does not track accuracy. The two largest on-disk footprints, HIPPORAG and A-MEM, bracket the accuracy spectrum: HippoRAG sits on the Pareto front while A-Mem is dominated everywhere. HINDSIGHT reaches the highest accuracy with a smaller footprint, and MEMGPT’s compact 0.69 GB store yields mid-pack accuracy.

The agent-memory paradigm is itself Pareto-dominated by raw retrieval. BM25’s ingestion is a one-time inverted-index build with no LLM or embedding calls (effectively \$0), yet it reaches 43.2% cross-domain accuracy, strictly Pareto-dominating four of the five agent memory systems and surpassed only by HINDSIGHT. Combined with Section 4.4, this says the transformations memory systems perform during ingestion (*e.g.*, fact extraction, graph construction, note rewriting) do not, on average, repay their cost in retrievability beyond what raw text already provides.

4.4 Failure Analysis

We dissect the mechanism behind the gaps above by attributing each error to either *retrieval failure* (no gold supporting message) or *reasoning failure* (gold retrieved but the LLM answered incorrectly).

Retrieval, not reasoning, is the bottleneck (Q1, Q2). Across all baselines, retrieval failures account for 41–79% of questions while reasoning failures stay narrowly bounded (7–24%): given the gold message, current LLMs reliably synthesise a correct answer. Most baselines lie on $y=x$ in Figure 5(Right), so accuracy scales near-linearly with recall. Two diagnostic deviations: HINDSIGHT on Finance lies above the diagonal as its paraphrased notes aggregate context across messages, while MEMGPT sits clearly below, retrieving adequately yet answering far worse, the signature of a representation that degrades downstream reasoning. This localizes the Q1/Q2 gaps to ingestion: structure and speaker context are lost before the LLM ever sees them.

Lexical shifts are the only failure that survives retrieval (Q3). Factoring out retriever quality (Figure 6), temporal is uniformly high, effectively solved given retrieval, whereas *term_ambiguity* stays below 40% across nearly every cell, the only type where representation, not retrieval, is the wall. BM25, which feeds the LLM raw message text, attains the highest mean conditional accuracy in both domains, matching or exceeding every memory-augmented baseline. The transformations performed by memory systems therefore tend to *reduce* rather than enhance the LLM’s ability to reason from retrieved evidence.

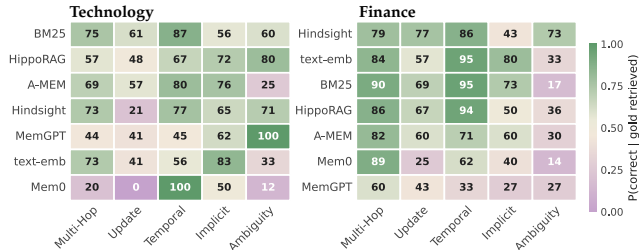


Figure 6: $P(\text{correct} \mid \text{gold retrieved})$ per (baseline, question type). Factoring out retriever quality isolates each memory representation’s reasoning ability.

5 Conclusion

We introduced **GroupMemBench**, a benchmark for agent memory in multi-party conversations. Our evaluation indicates that leading memory systems collapse across the benchmark and a bare BM25 baseline matches or exceeds most of them, showing that current ingestion pipelines erase the important features (*e.g.*, threaded conversation structure, lexical shifts) that group memory fundamentally depends on. Our finding highlights the need for more generalized agent memory systems for both dyadic and multi-party memory settings, and **GroupMemBench** drives valuable future improvements in multi-party memory solutions.

References

- [1] Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. Openclaw-rl: Train any agent simply by talking. *arXiv preprint arXiv:2603.10165*, 2026.
- [2] Renjun Xu and Yang Yan. Agent skills for large language models: Architecture, acquisition, security, and the path forward. *arXiv preprint arXiv:2602.12430*, 2026.
- [3] Sheryl Wei Ting Ng and Renwen Zhang. Trust in ai chatbots: A systematic review. *Telematics and Informatics*, 97:102240, 2025.
- [4] Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, et al. Personal llm agents: Insights and survey about the capability, efficiency and security. *arXiv preprint arXiv:2401.05459*, 2024.
- [5] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6):186345, 2024.
- [6] Jess Stratton. An introduction to microsoft copilot. In *Copilot for Microsoft 365: harness the power of generative AI in the Microsoft apps you use every day*, pages 19–35. Springer, 2024.
- [7] Wei-Chieh Huang, Weizhi Zhang, Yueqing Liang, Yuanchen Bei, Yankai Chen, Tao Feng, Xinyu Pan, Zhen Tan, Yu Wang, Tianxin Wei, et al. Rethinking memory mechanisms of foundation agents in the second half. *arXiv preprint arXiv:2602.06052*, 2026.
- [8] Chris Latimer, Nicol   Boschi, Andrew Neeser, Chris Bartholomew, Gaurav Srivastava, Xuan Wang, and Naren Ramakrishnan. Hindsight is 20/20: Building agent memory that retains, recalls, and reflects. *arXiv preprint arXiv:2512.12818*, 2025.
- [9] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [10] Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, et al. Memoryarena: Benchmarking agent memory in interdependent multi-session agentic tasks. *arXiv preprint arXiv:2602.16313*, 2026.
- [11] Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, et al. Ama-bench: Evaluating long-horizon memory for agentic applications. *arXiv preprint arXiv:2602.22769*, 2026.
- [12] Qingyao Ai, Yichen Tang, Changyue Wang, Jianming Long, Weihang Su, and Yiqun Liu. Memorybench: A benchmark for memory and continual learning in llm systems. *arXiv preprint arXiv:2510.17281*, 2025.
- [13] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-memeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813*, 2024.
- [14] Chris Frith and Uta Frith. Theory of mind. *Current biology*, 15(17):R644–R645, 2005.

- [15] Hyunwoo Kim, Melanie Sclar, Xuhui Zhou, Ronan Bras, Gunhee Kim, Yejin Choi, and Maarten Sap. Fantom: A benchmark for stress-testing machine theory of mind in interactions. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 14397–14413, 2023.
- [16] Kazutoshi Shinoda, Nobukatsu Hojo, Kyosuke Nishida, Saki Mizuno, Keita Suzuki, Ryo Masumura, Hiroaki Sugiyama, and Kuniko Saito. Tomato: Verbalizing the mental states of role-playing llms for benchmarking theory of mind. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 1520–1528, 2025.
- [17] Herbert H Clark and Susan E Brennan. Grounding in communication. 1991.
- [18] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [19] Bernal J Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems*, 37:59532–59569, 2024.
- [20] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13851–13870, 2024.
- [21] Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions. *arXiv preprint arXiv:2507.05257*, 2025.
- [22] Chuanrui Hu, Tong Li, Xingze Gao, Hongda Chen, Dannong Xu, Yi Bai, Tianwei Lin, Xinda Zhao, Xiaohong Li, Jiaqi An, et al. Evermembench: Benchmarking long-term interactive memory in large language models. *arXiv preprint arXiv:2602.01313*, 2026.
- [23] Wanjun Zhong, Lianhong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 19724–19731, 2024.
- [24] Zeyu Zhang, Quanyu Dai, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model-based agents. *ACM Transactions on Information Systems*, 43(6):1–47, 2025.
- [25] Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, et al. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8416–8439, 2025.
- [26] Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao, Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao Vijjini, Jiashu He, Hanchao Yu, et al. Personamem-v2: Towards personalized intelligence via learning implicit user personas and agentic memory. *arXiv preprint arXiv:2512.06688*, 2025.
- [27] Dongming Jiang, Yi Li, Guanpeng Li, and Bingzhe Li. Magma: A multi-graph based agentic memory architecture for ai agents. *arXiv preprint arXiv:2601.03236*, 2026.
- [28] Charles Packer, Vivian Fang, Shishir_G Patil, Kevin Lin, Sarah Wooders, and Joseph_E Gonzalez. Memgpt: towards llms as operating systems. 2023.
- [29] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *arXiv preprint arXiv:2502.12110*, 2025.
- [30] Haoran Tan, Zeyu Zhang, Chen Ma, Xu Chen, Quanyu Dai, and Zhenhua Dong. Membench: Towards more comprehensive evaluation on the memory of llm-based agents. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19336–19352, 2025.

- [31] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023.
- [32] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

Appendix

A Graph Schema

Node types and attributes. The synthesis graph $\mathcal{G}$ contains four structural node types created during graph construction and one dynamic node type added during message generation:

- **Domain** (d): a project. Stores the project’s category tags (e.g., *Quality Control*, *Compliance*), which are used to induce `related_to_project` edges between sibling projects.
- **Topic** (τ): a thematic decomposition of a domain. Topic identifiers are namespaced as `$d::topic_label$` to avoid collisions across projects.
- **Phase** (ϕ): a temporally bounded sub-problem of a topic. Each phase carries (i) a *progress category* drawn from {RISKS, MILESTONE, WORK ITEMS}, (ii) a *status goal* drawn from a category-specific schema (e.g., DETECTED/MITIGATED/ESCALATED for Risks), (iii) a *start* and *target* date used for timestamp generation and progress tracking, (iv) a *phase owner*, and (v) a synthetic documentation URL referenced by some prompts.
- **User** (u): a participant. Each user is assigned a persona $C_u = (\text{role}, \text{tone}, \text{style}, \text{expertise})$ where *tone* and *style* are drawn jointly *without replacement* from the Cartesian product of the two vocabularies, so every user has a distinct voice. The full vocabularies are listed in Table 3.
- **Message** (m): added at generation time. Stores the rendered text, the author, the timestamp, and the *path type* (`post`, `reply`, or `cross_project_reply`) used to produce it.

Edges. The full set of static and dynamic relations is given in Table 4. The two relations that most strongly shape the synthetic discourse are `active_in_phase` (which restricts the candidate authors of any message to the phase’s assigned members) and `related_to_project` (which is added between any two domains that share at least one category tag and which is the only edge that the cross-project handler walks across).

Persona vocabularies. Each user is assigned a persona along four axes: *tone*, *style*, *expertise*, and *role*. The vocabularies for the first three axes are domain-agnostic and listed in Table 3; the *role* vocabulary is domain-specific and drawn from the pool defined for each project family in our configuration (e.g., for Manufacturing: *Production Manager*, *Quality Engineer*, *Maintenance Engineer*, *Supply Chain Manager*, *Project Manager*). To ensure that every user has a recognisable individual voice, (*tone*, *style*) pairs are sampled *jointly without replacement* from the Cartesian product of the tone and style vocabularies, while expertise and role are sampled independently and uniformly. The tone, style, and expertise values do more than label the user: each value is paired with a concrete behavioural descriptor (e.g., the *anecdotal* style instructs the model to ground arguments in past experience without inventing project codenames) that is injected verbatim into the system prompt during message generation, as described in Appendix C.

B Path Sampling and Graph Walks

At each generation step we first sample a path type from the mixture $(p_{\text{post}}, p_{\text{reply}}, p_{\text{cross}}) = (0.20, 0.60, 0.20)$, with two deterministic overrides: (i) phases that have not yet received any message are forced into the `post` branch, and (ii) a phase whose recent posts are all open questions without engagement (`streak` ≥ 3 , see “question cooldown” in Appendix D) is forced into the `reply` branch. The chosen handler then walks $\mathcal{G}$ as follows.

Table 3: Persona vocabularies. Tone and style values are paired by sampling without replacement so users have distinct $(tone, style)$ combinations.

Attribute	Values
Tone	direct, casual, professional, empathetic, persuasive, humorous, skeptical, enthusiastic, anxious
Style	concise, chatty, elaborative, boring, action-oriented, anecdotal
Expertise	novice, intermediate, expert, specialist
Role	domain-specific (e.g., for Manufacturing: Production Manager, Quality Engineer, Maintenance Engineer, Supply Chain Manager, Project Manager)

Table 4: Edge relations in the synthesis graph $\mathcal{G}$. “Static” edges are created during graph construction; “dynamic” edges are added incrementally as messages are generated.

Relation	Source $\rightarrow$ Target	Type	Purpose
has_topic	$d \rightarrow \tau$	static	domain owns topic
has_phase	$\tau \rightarrow \phi$	static	topic decomposes into phases
belongs_to_project	$\phi \rightarrow \tau$	static	inverse pointer for traversal
involved_in	$u \rightarrow d$	static	user is on the project
has_member	$d \rightarrow u$	static	inverse member edge
active_in_phase	$u \rightarrow \phi$	static	user is active in the phase
has_phase_member	$\phi \rightarrow u$	static	inverse phase-membership
related_to_project	$d \rightarrow d'$	static	shared category bridge (cross-project)
authored_by	$m \rightarrow u$	dynamic	message authorship
posted_in	$m \rightarrow \phi$	dynamic	message anchored in phase
reply_to	$m \rightarrow m'$	dynamic	threaded reply edge

Channel post ($d \rightarrow \tau \rightarrow \phi \rightarrow u$). Domains with no messages so far are sampled with probability 0.8 to ensure breadth coverage; otherwise a domain is chosen uniformly. From the domain we walk a uniform has_topic edge, then a has_phase edge weighted by the *post-need* score

$$w_{\text{post}}(\phi) = \begin{cases} 10.0 & \text{if } \phi \text{ has no messages,} \\ 8.0 & \text{if } \phi \text{ has no posts,} \\ \max(1.0, 5.0 - 1.5 n_{\text{posts}}(\phi)) + 2 \sin(\pi p(\phi)) & \text{otherwise,} \end{cases}$$

where $p(\phi) \in [0, 1]$ is the phase’s temporal progress. The author u is then sampled uniformly from the phase’s active_in_phase members. The post type is conditioned on the existing message count: {kickoff, milestone} for empty phases, {question, update, coordination} at low volume, and the full set {update, question, blocker, decision, milestone, escalation, coordination} thereafter.

Threaded reply ($d \rightarrow \tau \rightarrow \phi \rightarrow m \rightarrow u$). A domain is sampled inversely proportional to its current message count so that high-volume projects do not monopolise replies. Within the domain we walk to a phase under the *phase-heat* score

$$w_{\text{heat}}(\phi) = 1 + 0.5 \min(n, 10) + 3.0 n_{\text{unanswered}} + 2 \sin(\pi p(\phi)),$$

where $n_{\text{unanswered}}$ counts posts that have received no reply. The parent message m is then chosen from the phase’s history with weight

$$w(m) = \underbrace{(1 + r_m)^{1.3} \cdot \mathbb{I}[r_m < 4 \vee 0.35]}_{\text{preferential attachment + saturation}} \cdot \underbrace{\max(0.15, 1 - \frac{a_m}{20})}_{\text{recency decay}} \cdot \underbrace{(2.0 \text{ if } m \text{ is a post else } 1.0)}_{\text{post-over-reply bias}},$$

where r_m is the current reply count of m and a_m its age in messages. The responder is drawn uniformly from active phase members excluding the parent author and the last three speakers when possible.

Cross-project bridge ($u \rightarrow \phi \rightarrow \tau \rightarrow d \xrightarrow{\text{rel}} d' \rightarrow \tau' \rightarrow \phi' \rightarrow u'$). We anchor at a uniformly sampled user u , follow their active_in_phase edge, climb back through topic and domain, cross exactly

one `related_to_project` edge to a sibling project d' , and descend to a target phase ϕ' in d' chosen by the *active-thread* score $w_{\text{thread}}(\phi) = 1 + 0.5 \min(n_\phi, 15)$. The target user u' is preferentially someone with the same role as u in ϕ' , and the walk is rejected if ϕ' starts strictly before ϕ (to keep the temporal direction of cross-project influence consistent).

Situational context assembly. After the path is fixed we assemble the situational context $\mathcal{S}_u$ by concatenating: (i) the most recent $W = 50$ messages of the target phase, optionally compressed by sampling 3 short snippets from any older window; (ii) the phase memory artefact described in Appendix D (sub-issue + ledger); and, with probability 0.25, (iii) a one-sentence cross-reference snippet drawn uniformly from the recent history of a sibling phase in the same project. $\mathcal{S}_u$ is then deduplicated against itself with a sequence-matcher threshold of 0.50 to suppress paraphrastic echoes.

C Prompt Templates

All prompts share a common *system* prompt that fixes the speaker’s voice, and one of four *user* prompt templates that fixes the conversational scenario. Templates are abbreviated below; full strings, including the persona-conditioned behavioural descriptors injected into the VOICE block, are released with the code.

System prompt (per-user, persona-conditioned).

```
You are a {role} on a Microsoft Teams channel.
VOICE: tone={tone_desc}; style={style_desc};
expertise={expertise_desc}.
LENGTH: replies 1-3 sentences ( $\leq 15$  words each); posts 2-4 sentences.
Never write paragraphs unless asked.
FORMAT: {style-specific formatting rule}.
RULES: stay in character; vary openings; never repeat a previous
opening or anecdote; do not parrot the last few messages.
```

Channel-post template (T_{post}).

```
BACKGROUND: {user, persona, enriched phase background}.
PHASE: {phase_name} ({stage} stage), status goal={status},
target={date}; stage guidance={...}.
TASK: {post_type_purpose} (kickoff / update / question / blocker /
decision / milestone / escalation / coordination), in the context of
{post_focus} for the {phase_name} phase.
VOICE: tone, style, expertise descriptors.
CONSTRAINTS: 2-4 sentences; pick one concrete thing; do not
announce phase start or percentage complete; phrasing rules (banned
phrases / openings, length target, optional messiness instruction);
optional doc-URL instruction.
```

Threaded-reply template (T_{reply}).

```
BACKGROUND, PHASE: as above.
CHANNEL HISTORY: {situational context  $\mathcal{S}_u$ }.
TASK: write a reply to {parent_message} in the {project} channel;
build on what has been said, address the original question, push
toward closure if late in the phase.
VOICE, MENTION: optional @{parent_author}.
CONSTRAINTS: phrasing rules; with probability 0.30 a hard
“ULTRA-SHORT 1-6 words” override; with probability 0.25 a mandatory
cross-reference to a sibling phase; closure instruction emitted when
phase progress  $> 0.8$ .
```

Cross-project role-to-role template (T_{cross}).

```
BACKGROUND: {user, persona, source phase}.
RECIPIENT ROLE: same as author’s role.
```

ORIGINAL POST: {first_post_in_target_phase}.
 THREAD CONTEXT: { S_u from target phase}.
 TASK: bring an experience-based perspective from the source project;
 share a transferable insight, ask a thoughtful question, or suggest
 an approach that helps move the target phase toward its status goal;
 reference earlier messages naturally.
 VOICE, phrasing rules, optional doc-URL, closure instruction.

Disagreement template (T_{disagree} , used in 25% of replies).

TASK: write a reply that pushes back on a point in the conversation;
 choose ONE of: challenge feasibility, question assumptions,
 propose alternative, flag overlooked risk, push back on scope, cite
 counter-evidence. The disagreement must be substantive (not “I
 agree but...”).

Noise template (T_{noise} , used with prob. 0.05).

TASK: write a message that contains exactly one realistic mistake,
 drawn uniformly from {wrong detail, misunderstanding, stale info,
 off-topic drift, premature assumption, wrong audience}. The
 mistake must be *subtle*; do not signal uncertainty; write with full
 confidence.

Phase-memory prompts. Two auxiliary prompts are issued to the same model independently of the message-generation prompts: (i) a *sub-issue prompt* called once per phase asking for $K=6$ distinct, non-overlapping sub-issues a real cross-functional team would surface, and (ii) a *ledger prompt* called every $L=8$ new messages that rewrites a ≤ 180 -word summary into three sections, DECIDED / OPEN QUESTIONS (with repeat counts) / PENDING COMMITMENTS, and is required to migrate any newly answered question from OPEN to DECIDED.

D Hyperparameters and Realism Knobs

Table 5 lists the values used to produce the corpus described in this paper. All values are taken from the released code and are exposed as a single POLICY dict so that they can be re-tuned in one place.

E Evaluation Rubrics for Synthetic Conversation

For all synthetic group chat data, we report quality using six core metrics, each scored by LLM-as-Judge protocols (e.g., G-Eval) on a scale from 1 to 5, with higher values indicating better performance:

- **Naturalness:** Measures how authentic and human-like the conversation appears. High scores indicate messages resemble real workplace communication, with natural phrasing and plausible imperfections.
- **Coherence:** Assesses the logical flow and connectivity between messages. Coherent conversations maintain topic continuity and clear transitions, avoiding abrupt or confusing shifts.
- **Diversity:** Captures the variety in communication styles, topics, and participant contributions. High diversity reflects a mix of message lengths, tones, and perspectives, avoiding repetitive exchanges.
- **Contextual Relevance:** Evaluates how well each message relates to the shared context and ongoing discussion. Relevant conversations consistently reference prior messages, project details, and team objectives.
- **Momentum:** Measures the degree to which the conversation progresses toward goals or resolutions. High momentum indicates that messages drive the discussion forward, address blockers, and facilitate decision-making.
- **Engagingness:** Assesses how interactive and stimulating the conversation is for participants. Engaging conversations feature active participation, questions, acknowledgments, and responses that encourage further dialogue.

Table 5: Generation hyperparameters.

Group	Parameter	Value
Path sampling	$p_{\text{post}} : p_{\text{reply}} : p_{\text{cross}}$	0.20 : 0.60 : 0.20
	Empty-phase post override probability	0.80
	Force-reply on question streak	≥ 3
	Reply target post bias	$\times 2.0$
	Reply target preferential-attachment exponent	1.3
	Reply target recency window / floor	20 msgs / 0.15
Context window	Recent verbatim window W	50 messages
	Older-context summary snippets	3
	Deduplication threshold	0.50
Realism perturbations	Noise rate	0.05
	Disagreement rate (replies only)	0.25
	Stylistic-imperfection rate	0.20
	Ultra-short reply rate	0.30
	Tone jitter rate	0.10–0.20
	URL inclusion rate	0.10 (post: 0.35)
	Cross-reference rate	0.25
	Forced decision-change eligibility	$0.3 \leq p(\phi) \leq 0.8$ and ≥ 4 msgs
Phase memory	Sub-issues per phase	6
	Sub-issue steering rate	0.70
	Ledger refresh interval	8 new messages
	Ledger context window	16 messages
Timestamps	Phase duration	10 days
	Inter-arrival distribution	exponential (Poisson process)
	Late-stage message cap	10 msgs after $p(\phi) \geq 0.95$
Corpus scale	Target messages (full run)	30,000
	Topics per project	3
	Phases per topic	5
	Users per project	4 (out of 6–10)
	Backbone language model	GPT-5

- **Overall Avg.:** The arithmetic mean of the above metrics, providing a single summary score for overall conversation quality.

These metrics collectively capture the realism, effectiveness, and collaborative dynamics of synthetic workplace conversations, enabling nuanced evaluation and comparison across datasets.

F Dataset Statistics

Table 6 reports per-domain corpus statistics for the synthesized GROUPMEMBENCH conversations. Each domain is realized as a collection of *projects* (one Teams-style channel per project), which are in turn decomposed into topics and time-bounded phases following the graph schema of Section 3.2. Within every phase, individual messages are tagged with a *path type* (*post*, *reply*, or *cross-project reply*), and may additionally be flagged as a *noise* or *decision* message.

Three properties of the corpus are intentional. First, the four domains are **matched on message count** (30,000 messages per domain), so any cross-domain difference in downstream accuracy or cost cannot be attributed to corpus-size imbalance. Second, the per-path-type proportions are stable across domains and closely track the path-sampling mixture $(p_{\text{post}}, p_{\text{reply}}, p_{\text{cross}}) = (0.20, 0.60, 0.20)$: posts and replies remain near a 1:3 ratio, with an additional $\sim 10\%$ of messages crossing project boundaries. Third, the participant and role counts vary by design, ranging from a **leaner** Healthcare setting (6 participants, 4 roles) to a richer Technology setting (18 participants, 6 roles), so memory-evaluation tasks see a spectrum of organizational complexity rather than a single homogeneous setting.

Table 6: Per-domain statistics of the synthesized group conversations.

Attribute	Technology	Finance	Healthcare	Manufacturing
# Projects	7	6	10	10
# Topics	35	30	30	30
# Phases	74	66	79	76
# Participants	18	12	6	9
# Roles	6	7	4	5
# Posts	6,599	6,764	7,043	6,816
# Replies	19,897	19,722	20,099	20,048
# Cross-project replies	3,504	3,514	2,858	3,136
# Noise messages	1,503	1,535	1,510	1,542
# Decision points	20,788	19,920	10,301	7,681
# Decision changes	74	66	113	99
Total messages	30,000	30,000	30,000	30,000

Table 7: Per-domain accuracy (%) on the **Technology** subset of GroupMemBench. Best per column in **bold**, second-best underlined.

Method	Multi-Hop	Update	Ambiguity	Implicit	Temporal	Abstention	Average
<i>RAG-based Methods</i>							
BM25	43.90	<u>30.60</u>	23.30	35.70	73.00	78.60	46.00
text-embedding-3-large	31.70	25.00	20.90	46.40	32.40	<u>82.10</u>	37.10
GraphRAG [32]	19.50	19.40	16.30	17.90	5.40	78.60	23.90
<i>Agent Memory Systems</i>							
Mem0 [9]	7.30	0.00	11.60	17.90	24.30	89.30	22.10
MemGPT [28]	24.39	25.00	16.28	35.71	16.22	75.00	29.58
A-Mem [29]	<u>36.60</u>	33.30	25.60	53.60	24.30	67.90	38.00
HippoRAG [19]	26.80	33.30	<u>27.90</u>	<u>50.00</u>	29.70	78.60	38.50
Hindsight [8]	<u>36.60</u>	11.10	37.20	<u>50.00</u>	<u>54.10</u>	71.40	<u>41.80</u>

G Per-Domain Results

Tables 7–10 report the per-domain accuracy of each baseline on the four domains of GroupMemBench (Technology, Finance, Healthcare, Manufacturing). The cross-domain micro-averages summarized in Table 2 are computed over the union of the filtered evaluation set across these four domains.

H Per-Domain Ingestion Cost and Storage

Table 11 reports the per-domain ingestion cost (USD) and on-disk storage (GB) summarized in the bubble plot of Figure 4. Costs are computed from per-call token counts collected by a LiteLLM proxy and converted to USD using OpenAI list prices (\$0.15 / \$0.60 per 1M input/output tokens for gpt-4o-mini; \$0.13 per 1M tokens for text-embedding-3-large), summed across all successful LLM and embedding calls during ingestion. Query-time calls are excluded throughout. MEMGPT’s archival ingest issues only embedding calls, so its LLM input/output token contributions are zero by design. Storage is measured directly with `du -sb` on each baseline’s per-domain store directory; HINDSIGHT and MEMGPT share a single Postgres volume across domains (approximately 10.6 GB and 2.77 GB respectively), which we apportion to each domain in proportion to a per-domain bookkeeping counter the system itself reports—node + link counts for HINDSIGHT, archival passage counts for MEMGPT.

Table 8: Per-domain accuracy (%) on the **Finance** subset of GroupMemBench. Best per column in **bold**, second-best underlined.

Method	Multi-Hop	Update	Ambiguity	Implicit	Temporal	Abstention	Average
<i>RAG-based Methods</i>							
BM25	47.90	<u>34.40</u>	11.10	<u>53.30</u>	46.70	75.90	42.10
text-embedding-3-large	52.10	31.20	22.20	60.00	44.40	72.40	44.40
GraphRAG [32]	22.90	12.50	26.70	6.70	6.70	72.40	24.30
<i>Agent Memory Systems</i>							
Mem0 [9]	31.20	6.20	8.90	20.00	31.10	86.20	29.40
MemGPT [28]	41.67	18.75	28.89	26.67	17.78	<u>79.31</u>	34.58
A-Mem [29]	<u>56.20</u>	31.20	33.30	46.70	31.10	69.00	43.50
HippoRAG [19]	58.30	37.50	<u>40.00</u>	33.30	<u>48.90</u>	69.00	<u>49.10</u>
Hindsight [8]	50.00	31.20	44.40	33.30	64.40	79.30	51.90

Table 9: Per-domain accuracy (%) on the **Healthcare** subset of GroupMemBench. Best per column in **bold**, second-best underlined. The Ambiguity and Implicit columns include 0 % / 50 % ties across many baselines, reflecting the small per-cell sample size of those question types in this domain.

Method	Multi-Hop	Update	Ambiguity	Implicit	Temporal	Abstention	Average
<i>RAG-based Methods</i>							
BM25	<u>37.50</u>	<u>11.80</u>	0.00	100.00	45.90	89.50	<u>43.10</u>
text-embedding-3-large	27.10	17.60	14.30	<u>50.00</u>	10.80	68.40	26.90
GraphRAG [32]	2.10	5.90	0.00	<u>50.00</u>	2.70	57.90	11.50
<i>Agent Memory Systems</i>							
Mem0 [9]	12.50	5.90	0.00	<u>50.00</u>	0.00	73.70	16.90
MemGPT [28]	12.50	17.60	0.00	0.00	5.60	89.50	21.70
A-Mem [29]	22.90	<u>11.80</u>	0.00	<u>50.00</u>	10.80	68.40	23.80
HippoRAG [19]	35.40	17.60	0.00	100.00	<u>27.00</u>	73.70	35.40
Hindsight [8]	45.80	17.60	14.30	<u>50.00</u>	45.90	<u>78.90</u>	45.40

I LLM Judge Configuration and Reliability

All accuracy numbers in the main text are produced by a single LLM judge that compares each agent’s answer to a gold reference and emits a binary verdict. This appendix specifies the judge’s configuration, the prompt and output parsing rule, and a manual reliability check.

Judge configuration. The judge is GPT-5 served via the Azure OpenAI API (deployment gpt-5, API version 2024-02-15-preview), with `max_tokens = 2048` and `temperature = 1.0`. Each (question, gold answer, predicted answer) triple is judged once; we do not use self-consistency or multi-sample voting.

Judge system prompt. The system message instructs the model to act as a strict equivalence judge, allow paraphrastic matches, and emit a structured verdict:

```
You are a strict judge evaluating whether an agent’s answer matches
the gold answer for a question.
Consider paraphrases correct if they have the same meaning as the
gold answer.
First provide a brief reasoning paragraph. Then provide the final
judgment on a new line using the format:
Final: Correct
```

Table 10: Per-domain accuracy (%) on the **Manufacturing** subset of GroupMemBench. Best per column in **bold**, second-best underlined.

Method	Multi-Hop	Update	Ambiguity	Implicit	Temporal	Abstention	Average
<i>RAG-based Methods</i>							
BM25	31.10	13.60	0.00	0.00	55.80	72.70	<u>41.10</u>
text-embedding-3-large	<u>33.30</u>	13.60	27.30	0.00	39.50	<u>75.80</u>	39.90
GraphRAG [32]	4.40	13.60	<u>18.20</u>	0.00	7.00	57.60	18.40
<i>Agent Memory Systems</i>							
Mem0 [9]	35.60	<u>9.10</u>	27.30	25.00	9.30	78.80	32.90
MemGPT [28]	11.10	4.50	<u>18.20</u>	0.00	9.30	72.70	22.80
A-Mem [29]	24.40	0.00	<u>18.20</u>	0.00	25.60	66.70	29.10
HippoRAG [19]	35.60	<u>9.10</u>	<u>18.20</u>	0.00	11.60	78.80	32.30
Hindsight [8]	35.60	<u>9.10</u>	27.30	0.00	<u>53.50</u>	78.80	44.30

Table 11: Per-domain ingestion cost (USD) and on-disk storage (GB) for the six baselines used in the trade-off analysis, corresponding to the bubble plot in Figure 4. Costs aggregate all LLM and embedding calls during ingestion (query-time calls excluded). The **Avg.** column is the unweighted mean across the four domains.

Method	Ingestion Cost (USD)					Storage (GB)				
	Tech.	Fin.	Healthc.	Manuf.	Avg.	Tech.	Fin.	Healthc.	Manuf.	Avg.
Mem0 [9]	16.82	34.92	10.99	10.89	18.41	0.75	1.21	1.00	1.01	0.99
MemGPT [28]	0.54	0.56	0.32	0.31	0.43	0.69	0.69	0.69	0.69	0.69
HippoRAG [19]	9.32	10.01	7.66	7.55	8.64	6.94	6.98	4.45	4.68	5.76
A-Mem [29]	39.44	42.66	11.14	11.28	26.13	7.53	8.63	5.12	5.13	6.60
Hindsight [8]	21.81	51.04	27.95	27.60	32.10	5.26	5.35	3.34	3.04	4.25
GraphRAG [32]	15.26	15.82	9.63	9.32	12.51	3.29	3.24	2.56	2.90	3.00

or
Final: Incorrect

The user message of each call contains the question, gold answer, and the agent’s predicted answer concatenated under fixed headers.

Output parsing. The judge’s reasoning is free-form, so we extract the verdict deterministically from the structured suffix. The parser scans the response for the last line beginning with `Final:` or `Final answer:`, lower-cases the text after the prefix, and applies the following precedence:

1. If the suffix contains any of `incorrect`, `wrong`, or `not correct`, the verdict is **Incorrect**.
2. Otherwise, if the suffix contains `correct`, the verdict is **Correct**.
3. Otherwise, the verdict is recorded as **Unclear** and excluded from the accuracy denominator.

Negative phrases are checked first because `not correct` is a substring of the positive trigger; the implementation is in `eval_lib.py` (lines 146–152).

Reliability check. We manually re-examined 100 (question, gold answer, predicted answer, judge verdict) tuples sampled from the main evaluation run, covering all four domains and all six question types. The LLM judge agreed with our verdict on 99 of 100 items (99%). The single disagreement was a false negative in which the agent’s answer correctly contained the gold content but added extra material beyond what the gold specified; the judge interpreted the additional content as inconsistent with the gold answer and returned *Incorrect*, where we would have accepted the response. We did not

encounter any false positives in the sample, indicating that the judge errs slightly on the strict side rather than being lenient.

Implications for the main results. A 99% agreement rate on this sample, combined with the strict-side bias of the lone disagreement, means that the reported accuracies are if anything mild lower bounds on the true correctness rate. Because the bias is uniform across baselines (no system is favoured by the judge’s strictness), the cross-baseline ranking in Table 2 is unaffected.

J Worked example: information loss during memory ingestion

We trace one Finance/user-implicit question through all eight baselines—from BM25 (no semantic processing during ingestion) to graphrag (per-domain knowledge graph + community summaries)—to show concretely how an ingestion-time rewrite can either preserve or erase the very fields the question depends on. The original chat message that contains the answer is reproduced once; we then show each baseline’s top retrievals and the gpt-5 QA agent’s answer, grouped by whether that answer matches the gold.

Question (Finance, user-implicit)

Asking user: User_13 (Compliance Officer)

Question: “Who do I need aligned on formatting rules for the mitigation plan in the *Risk: Formatting Inconsistencies* phase?”

Gold answer: *Finance and Data Engineering.*

Ground truth in the conversation

(Msg_28294, written by the asker himself)

Author: User_13 (Compliance Officer) **Channel:** Financial Reporting Automation

Phase: Risk: Formatting Inconsistencies **Timestamp:** 2025-07-23T17:51:00

“Milestone check: we’ve officially detected the Formatting Inconsistencies risk—our reports have now achieved the jazz hands phase of compliance. At 52% complete, the job is no longer just mapping regulations; it’s making sure headings, dates, and tables stop playing costume changes between template, export, and downstream views.

*From my side, this is a useful turning point because consistency is now a control issue, not just a presentation nuisance. I’ll keep pressure on standardizing the spec, and **I need Finance and Data Engineering aligned on the same formatting rules** before this turns into an audit-season sequel nobody asked for.”*

Baselines that answered correctly. The three correct answers come from very different ingestion pipelines. What they share is that User_13’s request and named counterparties remain reachable at retrieval time—either because nothing was rewritten at all, or because the rewrite kept a speaker anchor.

BM25 (tim)

(no semantic processing during ingestion; raw message indexed verbatim)

Top retrieved passage (User_13’s actual message at rank 2):

```
[user=User_13 / speaker_role=Compliance Officer
/ phase_name=Risk: Formatting Inconsistencies /
timestamp=2025-07-23T17:51:00 / msg_node=Msg_28294]
Milestone check... I need Finance and Data Engineering aligned
on the same formatting rules...
```

Agent answer: “Finance and Data Engineering.”

Why it works: the answer literally appears in the retrieved passage, with all metadata intact (User_13, Compliance Officer, exact phase + timestamp).

text-embedding-3-large*(dense retrieval over raw messages)*

The dense retriever ranks several semantically similar messages from *other* users above the answer message. Msg_28294 itself appears only at **rank 7** of the top-10:

```
[1] User_7 / Data Analyst / Risk: Formatting Inconsistencies
[2] User_7 / Data Analyst / Risk: Formatting Inconsistencies
[3] User_13 / Compliance Officer / 2025-07-19 (Msg_1545)
...
[7] User_13 / Compliance Officer / 2025-07-23 (Msg_28294) ←
    answer here
```

Agent answer: “Finance and Data Engineering.”

Why it works: the gpt-5 agent reads the full top-10 context and surfaces the correct phrasing from rank 7. The pipeline survives because nothing was rewritten—it just relied on a longer effective window than BM25 did.

hindsight
*(anchor)**(LLM-rewritten “experience” that explicitly retains the speaker anchor)***Top retrieved memory:**

```
[type=experience / msg_node=Msg_28294 /
mentioned_at=2025-07-23T17:51 / phase=Risk: Formatting
Inconsistencies]
“User_13 plans to standardize formatting specifications
and needs alignment from Finance and Data Engineering on
formatting rules to avoid future audit issues. Involving:
User_13 (Compliance Officer).”
```

Agent answer: “Finance and Data Engineering.”

Why it works: hindsight’s ingestion compresses the original post but keeps the speaker (User_13) bound to the request and to the named counterparties. Compression is heavy, but the three load-bearing fields—who, what they need, from whom—all survive.

Baselines that answered incorrectly. The five incorrect answers fall into three distinct failure modes: *topic-stub compression* (mem0), *wrong-speaker shadowing* (a-mem), and *multi-speaker over-broadening* (hipporag, memgpt, graphrag).

mem0 *✗ Incorrect* *(LLM rewrites each message into an extracted “memory”—failure mode: topic-stub compression)*

The original Msg_28294 is *not* in the top retrievals. Three unrelated messages have all been collapsed by the rewriter into a near-identical topic stub:

```
[User_4 / IT Systems Lead / phase=Risk: Calculation
Discrepancy] ‘Phase is Risk: Formatting Inconsistencies’
[User_7 / Data Analyst / phase=Risk: Regulatory Compliance
Gap] ‘Phase is Risk: Formatting Inconsistencies’
[User_3 / Product Owner / channel=Sustainable Finance
Strategy] ‘Requesting senior alignment on final wording ...’
```

What was lost.

- The *speaker* (User_13) is no longer attached to the request.
- The *request itself* (“I need ... aligned”) is compressed away; only the topic label survives.
- The *counterparty* (Finance and Data Engineering) is gone.

Agent answer: “Legal, Risk, and Reporting teams.”

The agent fills the gap with a generic guess from compliance-officer priors because the retrieved passages no longer mention any specific counterparty.

a-mem × Incorrect (*failure mode: wrong-speaker shadowing*)

All three top-ranked memories are User_7's posts on the same topic; the asker's own message never surfaces:

- [1] Author: User_7 / Data Analyst / phase=Risk: Formatting Inconsistencies (early)
- [2] Author: User_7 / Data Analyst / phase=Risk: Formatting Inconsistencies (mid)
- [3] Author: User_7 / Data Analyst / phase=Risk: Formatting Inconsistencies (late)

What was lost. Speaker identity isn't physically erased (Author: User_7 is in every retrieved memory) but it *has been ignored at retrieval time*: similarity-search returned three near-duplicate posts about the same topic from a single louder speaker, and shadowed User_13's actual request entirely.

Agent answer: (empty)

The agent declines to answer because none of the retrieved User_7 posts name a counterparty for User_13.

hipporag × Incorrect (*failure mode: multi-speaker over-broadening, KG-backed*)

The personalized-PageRank traversal returns three different speakers' posts about the same risk:

- [1] User_12 (Compliance Officer): "...Please weigh in from Finance and Data Engineering ..."
- [2] User_4 (IT Systems Lead): "...cross-functional review: Finance, Data Engineering, QA, and template owners..."
- [3] User_12 (Compliance Officer): "...I need Finance and Engineering to confirm ..."

What was lost. The right entities (Finance, Data Engineering) *are* in the retrieval, but they are scattered across three *other* users' requests, each with slightly different counterparty lists. The agent unions the candidate sets rather than honoring the asker's specific request.

Agent answer: "Finance, Data Engineering, QA, and the report/template owners (with Compliance for sign-off)."

The judge marks this incorrect because the union goes beyond the gold answer.

memgpt × Incorrect (*failure mode: multi-speaker over-broadening, archival-passage retrieval*)

Like a-mem, memgpt's archival retrieval returns three User_7 / User_4 messages on the topic. Unlike a-mem, the agent extracts every counterparty term it sees and concatenates them all:

- [1] User_7 (Data Analyst, early phase)
- [2] User_7 (Data Analyst, late phase)
- [3] User_4 (IT Systems Lead, late phase)

Agent answer: "Finance, Operations, Reporting Owners, Compliance, Data Engineering, QA, and Template Owners."

Why it fails. Same root cause as hipporag: the asker's specific request was never retrieved, so the agent assembled a "who-has-ever-been-mentioned" list. The two correct names (**Finance**, **Data Engineering**) are in there, but they are diluted by five extras the gold answer doesn't include.

graphrag × **Incorrect** (*failure mode: community-summary abstraction*)

graphrag’s local search returns a community summary rather than any individual chat message:

```
[graphrag local context (per-domain index)]  
---Reports---  
Treasury Management and Compliance Community ... “centered  
around the critical roles of several key entities involved in  
financial ...”
```

What was lost. An entire layer of granularity. The community report aggregates dozens of messages (from many speakers) into a domain-level abstract; the asker, the specific request, and the named counterparties have all been folded into a topical narrative.

Agent answer: “**Finance** (including Finance Ops), QA, and **Data Engineering/Engineering**.”

The agent recovers the right entities but again over-broadens to a generic compliance-counterparty list.

Summary. This single question is enough to read off the design space: the *outcome* (correct or not) is set at *ingestion time*, before retrieval or QA gets a chance to fix anything. The two ingestion strategies that preserve the answer are, on the surface, opposites—BM25 keeps every original token and metadata field; hindsight rewrites aggressively but explicitly conditions every “experience” on its speaker. What unites them is that *User_13’s request and its named counterparty stay co-located* in some retrievable unit. Each of the five failures breaks that co-location in a different way: by collapsing the content to a topic label (mem0), by ranking similar-topic posts from a louder speaker above the asker’s own (a-mem), or by mixing several speakers’ counterparty lists into a single over-broad answer (hipporag, memgpt, graphrag). Section 4.2 quantifies the same pattern in aggregate; this appendix shows its mechanics on a single instance.

K Limitations

GroupMemBench currently focuses on English-language, text-only workplace conversations across four domains (Technology, Healthcare, Manufacturing, Finance). Two natural extensions are left to future work: a *multilingual* setting, where speaker-conditioned vocabulary and Theory-of-Mind effects interact with code-switching and translation drift; and a *multimodal* setting, where group chats also contain images, attachments, and voice notes, requiring memory systems to ground references across modalities. We expect both extensions to be additive on top of the graph-grounded synthesis pipeline introduced here, and the design principles of speaker-aware ingestion, audience-adapted language, and reply-structure control transfer directly to those settings.

L Broader Impacts

GroupMemBench is intended to drive research on agent memory in realistic multi-party settings, with potential positive impact on team collaboration tools such as channel assistants, project copilots, and meeting summarizers, where better speaker-grounded memory can reduce miscommunication and lower the cognitive load of long-running discussions. On the negative side, stronger speaker-conditioned memory could in principle support workplace surveillance or more targeted persuasion if irresponsibly deployed; we mitigate this at the artifact level by releasing only synthetic conversations grounded in fictional personas (Section 3.2) rather than any real chat logs, and we disclose the full persona vocabularies and prompt templates (Appendices A, C) so downstream users can audit and reweight before reuse.